TAR: Temporal Action Regularization for Smooth Action Chunking in Vision-Language-Action Models

Training-time boundary-velocity regularization for chunk-based VLAs — no architecture change, zero inference overhead

Ningqiu He and **Chi Kin Lam** (corresponding) Macao Polytechnic University, Macao, China P2625633@mpu.edu.mo · cklamsta@mpu.edu.mo

Poster size: A0 landscape (1189 × 841 mm)

Motivation

Modern VLAs predict **T = 8** future actions per forward pass (action chunking). On **LIBERO-10**, the official BitVLA long-horizon fine-tune reaches **86.4%** success but shows systematic **chunk-boundary discontinuities**.

3.1×	10×+	L1 only
Inter-chunk jump vs. intra-chunk smoothness (IB / S1)	Peak step velocity vs. average (MaxJump / S1)	Standard loss does not align consecutive chunks

Goal: encourage smooth hand-offs between independently predicted chunks without temporal ensembling at deployment.

Method — Temporal Action Regularization (TAR)

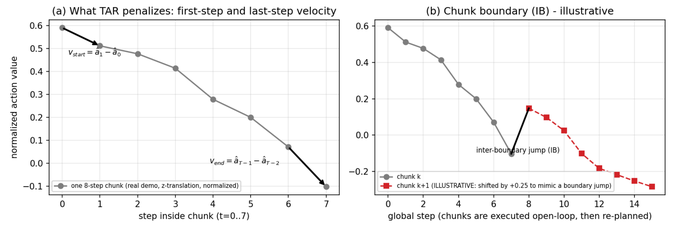
Given predicted chunk $\hat{a} \in \mathbb{R}^{B\times T\times D}$ (T = 8, D = 7 end-effector dims + gripper), augment L1 regression:

$$\mathcal{L} = \mathcal{L}_{L1} + \lambda_{TAR} \cdot \mathcal{L}_{TAR}$$

$$\mathcal{L}_{TAR} = (1/BD)\, \Sigma_{b,d}\, (|\hat{a}_{b,1,d} - \hat{a}_{b,0,d}| + |\hat{a}_{b,T-1,d} - \hat{a}_{b,T-2,d}|)$$

Intuition: each chunk should *start slowly* and *end slowly*, so the next chunk meets a compatible velocity. Only boundary steps are penalized — interior fast motions remain expressible.

~5 lines of code · **No new parameters** · **Same inference pipeline** · **BitVLA 3B · 1.58-bit · ~1.4 GB VRAM**



Chunking creates independent predictions at boundaries; TAR penalizes high boundary velocities during training.

Setup

- **Base model:** BitVLA (bf16) from `bitvla-bf16`
- **Benchmark:** LIBERO-10 (`libero_10_no_noops`)
- **Training:** full fine-tune, 10,001 steps, 4× RTX 4090D (FSDP)
- **Eval:** 500 episodes per checkpoint; smoothness from logged actions
- **Ablations:** $\lambda_{TAR} \in \{0, 0.01, 0.05, 0.10\}$

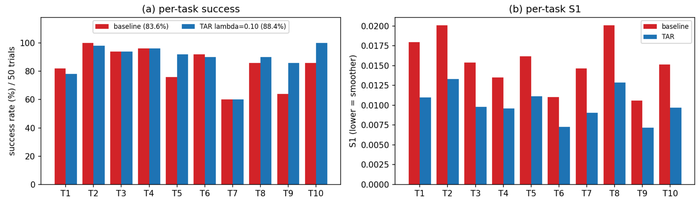
Metrics

Success rate ↑ · S1, MaxJump, InterBoundary, gripper switches ↓

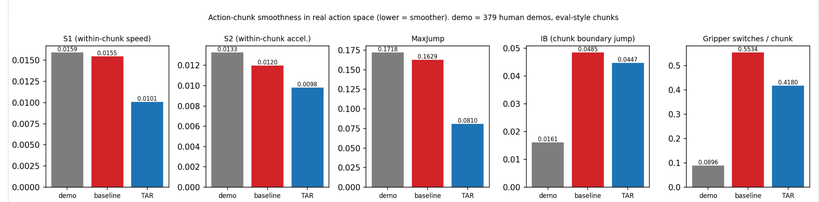
Main Results (λ ablation)

λ_{TAR}	Success	Δ vs baseline	S1 ↓	MaxJump ↓	InterBoundary ↓
0.00	86.4%	—	0.0154	0.1613	0.0477
0.01	89.0%	+2.6 pp	0.0111	0.0844	0.0421
0.05	90.8%	+4.4 pp	0.0104	0.0794	0.0451
0.10	**93.6%**	**+7.2 pp**	**0.0099**	**0.0761**	**0.0460**

Relative to baseline at λ = 0.10: **S1 −36%**, **MaxJump −53%**, **gripper switches −24%**. Success improves monotonically with λ in {0, 0.01, 0.05, 0.10}.



Per-task success: 9/10 tasks improve or tie; largest gains on multi-object placement.



Representative trajectories: smoother execution with TAR (λ = 0.10).

Takeaways

- **Problem:** chunk boundaries dominate motion discontinuities despite good intra-chunk predictions.
- **Fix:** lightweight boundary-velocity regularizer; plug-in for any chunk-based VLA trainer.
- **Evidence:** +7.2 pp success on LIBERO-10 with BitVLA; broad smoothness gains.
- **Insight:** stronger λ can act as implicit generalization regularization (higher train loss, best eval).

Limitations & next steps

- Single backbone / benchmark; transfer to OpenVLA-OFT, π_0, other chunk lengths T.
- Statistical replication & checkpoint release (ongoing cloud reproduce).

Code & reports: `bitvla-tar/` (hnqhnq/robotic monorepo)